

LATEX WRITESENSE AI

SHIVADA MANOJ P

MCA S4

TCR24MCA-2054

Main Project

Guided By:

Sreejith V P



Department of Computer Applications
Government Engineering College Thrissur Kerala



Outline

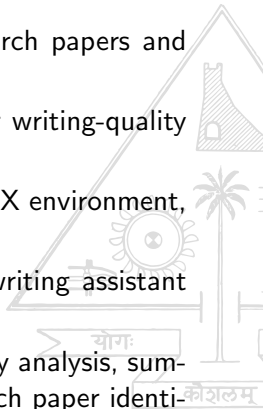
- 1 Introduction
- 2 Literature Review
- 3 Existing Systems
- 4 Proposed System
- 5 Requirements Specification
- 6 Input and Output
- 7 System Architecture
- 8 Data Flow Diagram
- 9 System Modules
- 10 System Modules
- 11 Implementation
- 12 Future Enhancements
- 13 Acknowledgement





Introduction

- Academic and technical writing requires accuracy, clarity, and coherence.
- Overleaf is widely used for LaTeX-based research papers and theses.
- However, Overleaf provides limited support for writing-quality improvement.
- Most AI writing tools operate outside the LaTeX environment, disrupting workflow and risking syntax errors.
- This project introduces a selection-based AI writing assistant integrated directly into Overleaf.
- The system provides grammar correction, clarity analysis, summarization, multilingual translation, and research paper identification.

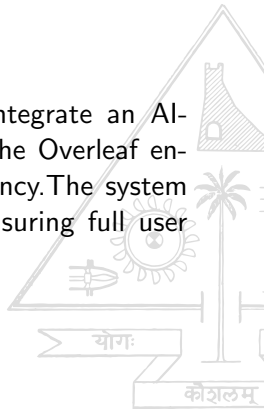




Introduction

Objectives:

The primary objective of this project is to integrate an AI-assisted writing support system directly into the Overleaf environment to enhance academic writing efficiency. The system preserves the LaTeX syntax integrity while ensuring full user control over all suggested modifications.





Literature Review

J. Hou, A. L. Huikai, N. Chen, Y. Gong, and B. He, “PaperDebugger: A Plugin-Based Multi-Agent System for In- Editor Academic Writing, Review, and Editing,”

- Integrated AI assistance directly into Overleaf
- Supported selection-based in-editor editing
- Provided LLM-driven writing and review support
- Eliminated external copy–paste workflows
- **Observation:** Shows the benefit of in-editor AI writing tools, but uses complex multi-agent architectures, motivating simpler and more controlled designs. [1]



Literature Review

S. Totawad, R. Patil, A. Thorat, D. Sasane, Y. D. Sinkar and S. Gaikwad, "Research Papers Recommendation System Prediction Using Deep Learning and LLMs,"

- Proposed an AI-based research paper recommendation system
- Used deep learning and LLM embeddings for semantic similarity
- Recommended papers based on content, citations, and user preference
- Improved research discoverability in large academic repositories
- **Observation:** Demonstrates the effectiveness of LLM-based paper recommendation, but operates as a standalone system, not integrated into the LaTeX writing workflow.[2]



Literature Review

Jovic, M., Papakonstantinidis, S. Kirkpatrick, R. "From redink to algorithms: investigating the use of large language models in academic writing feedback"

- Compared human feedback vs. LLM-generated feedback.
- Evaluated grammar, structure, tone, clarity in student writing.
- Showed that human feedback excels in contextual understanding, deeper reasoning, and personalized guidance
- Demonstrated that LLMs provide consistent and efficient surface-level feedback
- Reported limitations in contextual understanding and personalized guidance
- **Observation:** Highlights the need for controlled, transparent AI feedback



Literature Review Summary

Paper	Focus Area	Key Contributions and Observations
Hou et al. (2023) [1]	In-editor AI writing support	Integrated AI assistance directly into Overleaf with selection-based editing. Provided LLM-driven writing and review support while eliminating external copy-paste workflows.
Totawad et al. (2024) [2]	Research paper recommendation	Proposed an AI-based paper recommendation system using deep learning and LLM embeddings for semantic similarity. Recommended papers based on content, citations, and user preferences.
Jovic et al. (2023) [?]	LLM-based academic feedback	Compared human feedback with LLM-generated feedback across grammar, structure, tone, and clarity. Found that humans excel in contextual understanding and personalized guidance, while LLMs provide consistent and efficient surface-level feedback.

Table 1: Summary of Related Work



Existing Systems

- Overleaf mainly focuses on LaTeX document formatting and collaboration.
- No built-in support for grammar correction or clarity analysis.
- No readability evaluation or academic writing assistance.
- Dependence on external writing tools requiring manual copy-paste of LaTeX content.
- Workflow disruption when switching between the editor and external tools.
- Risk of LaTeX syntax breakage (commands, equations, references) during external processing.
- Lack of integrated features such as summarization, translation, and research paper identification.



Proposed System

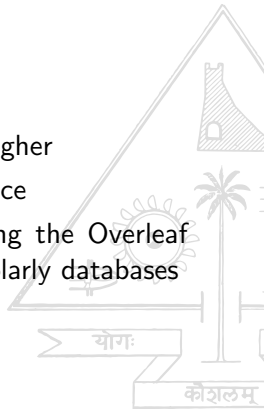
- Provides AI-assisted writing support directly within the Overleaf LaTeX editor.
- Eliminates manual copy–paste by integrating writing assistance into the editor.
- Performs grammar correction using a rule-based approach and AI-based.
- Analyzes writing clarity using readability metrics and AI evaluation.
- Generates summaries of selected text to support quick content understanding.
- Enables multilingual translation of academic text.
- Recommends relevant research papers using keyword-based scholarly search.
- Preserves LaTeX syntax and applies changes only with user approval.



Requirements Specification

Hardware Requirements

- **Memory:** Minimum 8 GB RAM
- **Processor:** Intel Core i5 / AMD Ryzen 5 or higher
- **Storage:** Minimum 20 GB free HDD/SSD space
- **Internet Connectivity:** Required for accessing the Overleaf editor and retrieving research papers from scholarly databases





Requirements Specification

Software Development Requirements

- **Programming Language:** Python 3.10
- **Framework:** FastAPI
- **Backend Server:** Uvicorn
- **Grammar Checking Tool:** LanguageTool
- **AI Model:** LLaMA 3
- **Translation Model:** NLLB (facebook/nllb-200-distilled-600M)
- **Research Paper Retrieval:** CrossRef API

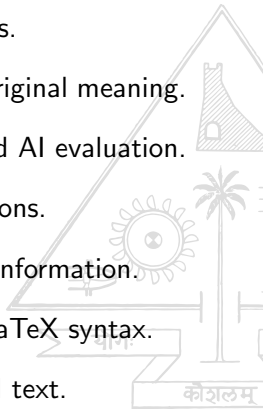




Requirements Specification

Functional Requirements

- Processes only user-selected sentences or paragraphs.
- Performs grammar correction while preserving the original meaning.
- Analyzes writing clarity using readability metrics and AI evaluation.
- Provides clarity feedback and improvement suggestions.
- Summarizes selected text without introducing new information.
- Supports multilingual translation while preserving LaTeX syntax.
- Identifies relevant research papers based on selected text.

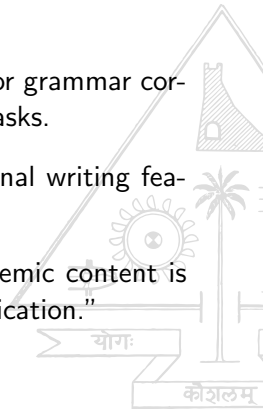




Requirements Specification

Non-Functional Requirements

- **Performance:** Provides acceptable response time for grammar correction, clarity analysis, and other text processing tasks.
- **Scalability:** Supports future extension with additional writing features and language models.
- **Security & Privacy:** Ensures that the user's academic content is handled safely through controlled backend communication."





Input and Output

Input

- User-selected text from a LaTeX document in the Overleaf editor

Output

- Corrected text with grammatical errors fixed
- Clarity analysis with readability metrics and AI feedback
- Concise summary of the selected text
- Translated text in the user-specified target language
- List of relevant research papers related to the selected text





System Architecture

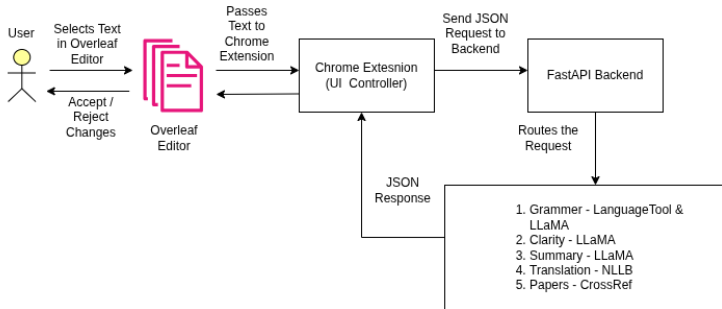


Figure 1: Architecture Diagram



Data Flow Diagrams

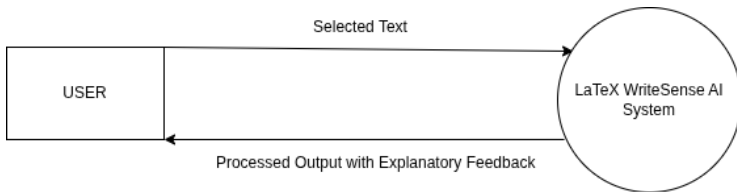


Figure 2: Level 0 DFD





Data Flow Diagrams

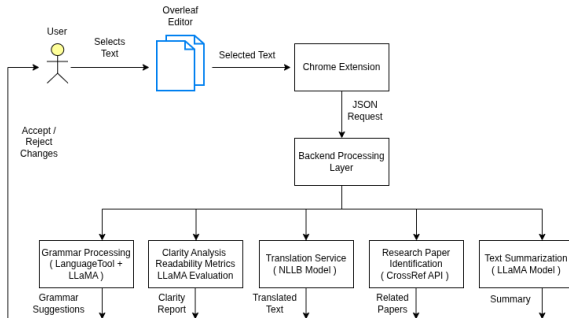


Figure 3: Level 1 DFD



System Modules

- **User Interface Module:** Provides user interaction through a Chrome extension integrated with the Overleaf LaTeX editor.
- **Text Selection & Input Handling Module:** Captures user-selected sentences or paragraphs from the LaTeX editor and prepares the text for processing.
- **Backend Request Management Module:** Receives requests from the extension, validates inputs, and routes them to appropriate backend services using FastAPI.
- **Grammar Correction Module:** Performs AI-Based, Rule-based grammatical analysis using LLaMA, LanguageTool and generates safe corrections without affecting LaTeX syntax.



System Modules

- **Clarity Analysis Module:** Evaluates academic writing quality using readability metrics and AI-based structural evaluation.
- **Summarization Module:** Generates concise summaries of selected academic text while preserving the original meaning.
- **Multilingual Translation Module:** Translates selected text into the target language while preserving LaTeX commands and structure.
- **Research Paper Identification Module:** Extracts research-related keywords and retrieves relevant academic papers from scholarly databases.
- **Result Presentation & User Control Module:** Displays results and suggestions to the user and allows changes to be applied only after user approval.



Implementation

- Successfully implemented the core processing pipeline of the **Write-Sense AI academic writing assistant** integrated with the Overleaf LaTeX editor.
- **Text Selection and Input Capture:** The Chrome extension captures user-selected text from the Overleaf editor and prepares it for processing.
- **Backend Request Routing:** Selected text is sent to FastAPI backend services where requests are validated and routed to appropriate processing modules.
- **Grammar Processing:** AI-Based, Rule-based grammar checking is performed using LLaMA and LanguageTool to detect errors and generate safe corrections without affecting LaTeX syntax.



Implementation (contd.)

- **Clarity Analysis:** Academic readability and structural quality are evaluated using metrics such as Flesch Reading Ease, lexical density, clause density, and passive voice ratio.
- **Summarization and Translation:** Selected text can be summarized or translated into multiple languages using transformer-based models while preserving LaTeX structure.
- **Research Paper Identification:** Relevant academic papers are retrieved from cross-ref database based on extracted keywords and research domain analysis.
- **Result Presentation:** All generated outputs are displayed in a dock-style interface within the Chrome extension, allowing the user to review and optionally apply the suggested changes.



Final Results

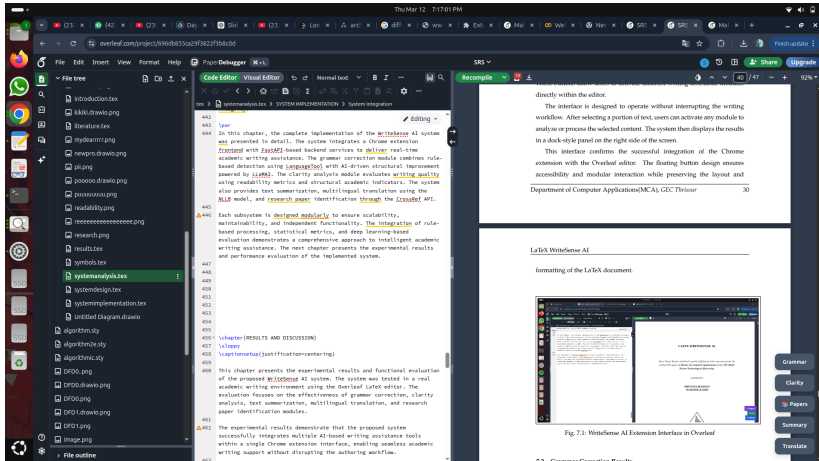


Figure 4: Interface



Final Results

WriteSense AI ×
from draft to distiction

Rule-Based Grammar (LanguageTool)

is be

Consider using either the present participle "is being" here.

Suggestion: is being

Rule Corrected Version

The project is being extended by incorporating richer graph features, evaluating additional graph neural network architectures, and exploring hybrid models combine graph-based representations with feature-based

[Apply Rule Fixes](#)

Figure 5: Grammar Correction





Final Results

Malware (2) - Online Let

File Edit Insert View Format Help Python Debugger

Code Editor Visual Editor

Normal text

SystemImplementation.py > SYSTEM IMPLEMENTATION > Dataset Loading and Organization

```
1 chapter(SYSTEM IMPLEMENTATION)
2 @property
3
4 """
5 System Implementation is the phase where the theoretical design and analytical framework are transformed into
6 a functional malware classification system. For the Graph-Based Malware Classification and Comparison system,
7 implementation involves loading graph-structured malware data, constructing node-level structural features,
8 developing a Graph Isomorphism Network (GIN) model for deep learning-based classification, extracting graph-level
9 statistical features for a Random Forest baseline model, and performing comprehensive evaluation. This phase
10 ensures that all components operate cohesively to provide accurate and reliable malware classification results. By
11 carefully implementing each module, the system effectively processes structural graph representations of software
12 samples and produces automated malware or benign predictions along with detailed performance analysis, making the
13 system suitable for academic research and practical security evaluation.
14
15 """
16
17 @section(Setup Development Environment)
18
19 """
20 An effective development environment is essential for implementing the malware classification system. Visual
21 Studio Code (VS Code) is used as the primary IDE. The system is implemented using Python 3.10, along with
22 libraries including PyTorch, PyTorch Geometric, NumPy, Pandas, Scikit-learn, NetworkX, and Registrator. GPU
23 acceleration is enabled using CUDA for efficient deep learning model training. All required dependencies are
24 installed using (tests) within a virtual environment to ensure compatibility and reproducibility.
25
26 """
27
28 @section(Dataset Loading and Organization)
29
30 """
31 The MalNet-Tiny dataset is used for experimentation. The dataset consists of gpg-extracted graph
32 representations of software samples. Each sample is stored as an edge list file representing structural
33 relationships within the program. During implementation, all graph files are parsed, converted into graph data
34 objects, and assigned corresponding class labels. The dataset is shuffled to prevent class-order bias and split
35 into training and testing sets.
36
37 """
38
39 @section(Graph Feature Construction)
40
41 """
42 Before training the deep learning model, node-level structural features are generated from each graph. Since
43 the dataset contains only structural connectivity information, node degree is computed and used as the primary
44 feature for each node. The edge lists are converted into edge index representations compatible with PyTorch
45 Geometric. These graph data structures include node features, edge indices, and class labels, enabling graph-level
46 learning. This preprocessing ensures that structural characteristics of each sample are effectively captured for
47 classification.
48
49 """
50
51 @section(Graph Isomorphism Network (GIN) Model)
52
53 """
```

Malware (2)

Recompile

WriteSense AI

Document Analysis

Primary Domain: Malware Detection

Sub Domains:

- Computer Security
- Machine Learning

Extracted Keywords:

- Control-Flow Graph
- Graph Isomorphism Network
- Random Forest
- Graph Neural Networks

Related Research Papers

Year Filter: 2019 - 2025

1. Malware Detection by Control-Flow Graph Level Representation Learning With Graph Isomorphism Networks

Yun Gao, Hiroaki Hasegawa, Yukio Yamaguchi, Hajime Shiraiwa (2022)

Open Paper

Citation (IEEE):

[1] Yun Gao, Hiroaki Hasegawa, Yukio Yamaguchi, et al., "Malware Detection by Control-Flow Graph Level Representation Learning With Graph Isomorphism Networks," *European Conference on Cyber Warfare and Security*, 2022.

2. Malware Detection Using Dynamic Graph Neural Networks

Purushoth Kumar, Stephen OShaughnessy (2025)

Open Paper

Citation (IEEE):

[1] Purushoth Kumar, Stephen OShaughnessy, "Malware Detection Using Dynamic Graph Neural Networks," *European Conference on Cyber Warfare and Security*, 2025.

3. Malware Detection Analysis Using Gated Graph Neural Networks and Graph Convolutional Networks

Node Butti Hanta Kusuma, Adhiti Sugenda Ginting (2025)

Figure 6: Research Paper Recommendation



Final Results

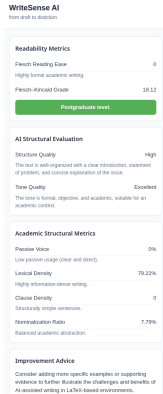


Figure 7: Clarity Analysis





Final Results

The screenshot displays a web browser window with a document titled "Malware Detection: A Graph and Random Forest Model". The document is written in English and discusses malware detection techniques, including Graph Isomorphism Networks (GIN) and Random Forests. The text is highlighted in blue, and a sidebar on the right, titled "WriteSense AI", offers a "Multilingual Translation" service. The sidebar includes a dropdown menu for selecting the target language (currently set to "Greek") and a "Translate" button. Below the button, there is a section for the translated text in Greek, which is currently blank. The sidebar also contains a "Close" button at the bottom.

Figure 8: multilingual Translation



Final Results

The screenshot shows a web browser window with the URL <https://overleaf.com/project/69e1d6b5b5c7482b666588>. The browser tabs include various documents and a 'Finish update' button. The main content area displays a document titled 'Malware (2)' with a 'WriteSense AI' sidebar on the right. The document content includes a literature review section with references to malware detection research. The sidebar shows a 'Summary' section with a brief overview of malware detection research.

Malware (2)

WriteSense AI
From draft to document

Summary

Malware detection has become a critical research area due to rapid growth in sophistication of malicious software. Traditional signature-based methods struggle with unknown or obfuscated variants, leading researchers to adopt machine learning approaches like static analysis and graph-based learning.

5.4 Graph Isomorphism Network (GIN) Model

The Graph Isomorphism Network (GIN) model is a graph-level multi-class classification. The architecture consists of GIN convolution layers followed by non-linear activation functions. A global mean pooling layer to aggregate node embeddings. A fully connected layer maps the learned embeddings to the final class predictions. The model is trained using the Adam optimizer. GPU acceleration improves training efficiency.

Department of Computer Applications(MCA), GEC Thane

Malware Detection: A Graph and Random Forest Comparison

5.5 Model Training and Evaluation

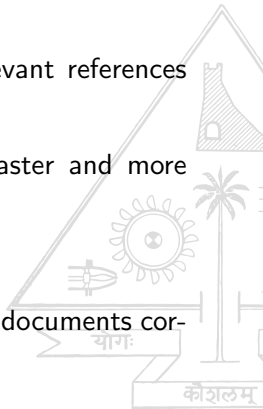
The dataset is divided into training and testing sets, used for training and 20% reserved for evaluation. The model is trained for multiple epochs, and performance metrics such as accuracy, and testing accuracy are monitored at each epoch. After training, the system generates a detailed classification report, including precision, recall, F1-score, and a confusion matrix.

Figure 9: Summarization



Future Enhancements

- Use advanced language models to improve grammar correction and make writing clearer.
- Provide citation suggestions to help users find relevant references while writing.
- Improve system performance to make responses faster and more efficient.
- Enable paraphrasing for better academic writing
- Recommend LaTeX commands to help users format documents correctly
- Expand platform support beyond Overleaf





Conclusion

- Successfully developed **WriteSense AI**, an AI-assisted academic writing support system integrated with the Overleaf LaTeX editor through a Chrome extension.
- Implemented a modular architecture combining a Chrome extension frontend with **FastAPI-based backend services** for text processing.
- Integrated rule-based grammar correction using **LanguageTool** and AI-based using LLaMA.
- Provided additional writing assistance features including **summarization, multilingual translation, and related research paper identification**.
- Designed the system to process only **user-selected text**, ensuring transparency, LaTeX syntax preservation, and full user control over document modifications.



References

- [1] J. Hou, A. L. Huikai, N. Chen, Y. Gong, and B. He, “PaperDebugger: A Plugin-Based Multi-Agent System for In-Editor Academic Writing, Review, and Editing,” arXiv preprint arXiv:2512.02589, 2025.
- [2] S. Totawad, R. Patil, A. Thorat, D. Sasane, Y. D. Sinkar and S. Gaikwad, “Research Papers Recommendation System Prediction Using Deep Learning and LLMs,” 2025 International Conference on Computing Technologies (ICOCT), Bengaluru, India, 2025
- [3] Jovic, M., Papakonstantinidis, S. & Kirkpatrick, R. “From red ink to algorithms: investigating the use of large language models in academic writing feedback” Lang Test Asia 15, 59 (2025). <https://doi.org/10.1186/s40468-025-00389-2>.



Thank You

